Project Report
Implementing PPO on Lunar Lander and Swimmer Environments

Introduction

My interactions with RL were limited to Value based algorithms, discrete action space environments and was only exposed to the theoretical concepts of Deep RL and Actor Critic approach. This was an effort to make my hands dirty by selecting a Policy based algorithm and trying to run it on a continuous action space environment while gaining practical knowledge of Deep RL algorithms. My curiosity also led to the question of the differences that come up while handling discrete and continuous action space environments. Out of the many available algorithms, PPO or Proximal Policy Optimization Algorithm was chosen for its ease of implementation and its flexibility of handling both kinds of action space environments. The paper published by John Schulman et al in 2017 titled Proximal Policy Optimization Algorithms was considered as a guide and is referred as the 'Original' in the rest of the report to develop my implementation along with various other blogs and papers which are listed at the end in References.

Initial Idea

The initial idea was to try and implement PPO on Lunar Lander Environment (discrete actions) as a sanity check, an environment which was already familiar and improve on it to handle even Swimmer (continuous actions) environment from the MuJoCo suite of environments from OpenAI's Gymnasium. Swimmer was particularly selected from MuJoCo as it is easier to implement among the others.

Swimmer-v5:
3 links connected with the help of 2 rotors act as the agent. The goal is to swim forward within 1000 steps after which the episode gets truncated. There are no terminal states, and the state space is 8. The action is the torque applied to the rotors which ranges from -1 to +1. The reward

is calculated as the difference of forward movement and the actions which moves the agent forward, this helps in penalizing heavy movements while incentivizing forward movement.

The plan was to develop an implementation that can run in both discrete and continuous action spaces and evaluating the policies developed to understand how the hyperparameters affect the policy. We evaluate the policy at every 5% level of total steps.

Among the different ways tried in the original, the clipping with an epsilon value of 0.2 was considered as the way forward to clip the surrogate objective function since it performed the best in their experiments.

What was done

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

The algorithm that was mentioned in the 'original' was modified slightly, namely:

1. A single actor was used unlike multiple actors.
2. The advantages were normalized as per the suggestion mentioned in another paper.
3. The entropy coefficient was annealed gradually as the timesteps increased, to reduce the level of exploration towards the end of our run.
   The way it was achieved is mentioned below.
   entropy_coef=
   entropy_coef_i-(steps/max_timesteps)*(entropy_coef_i-entropy_coef_e)

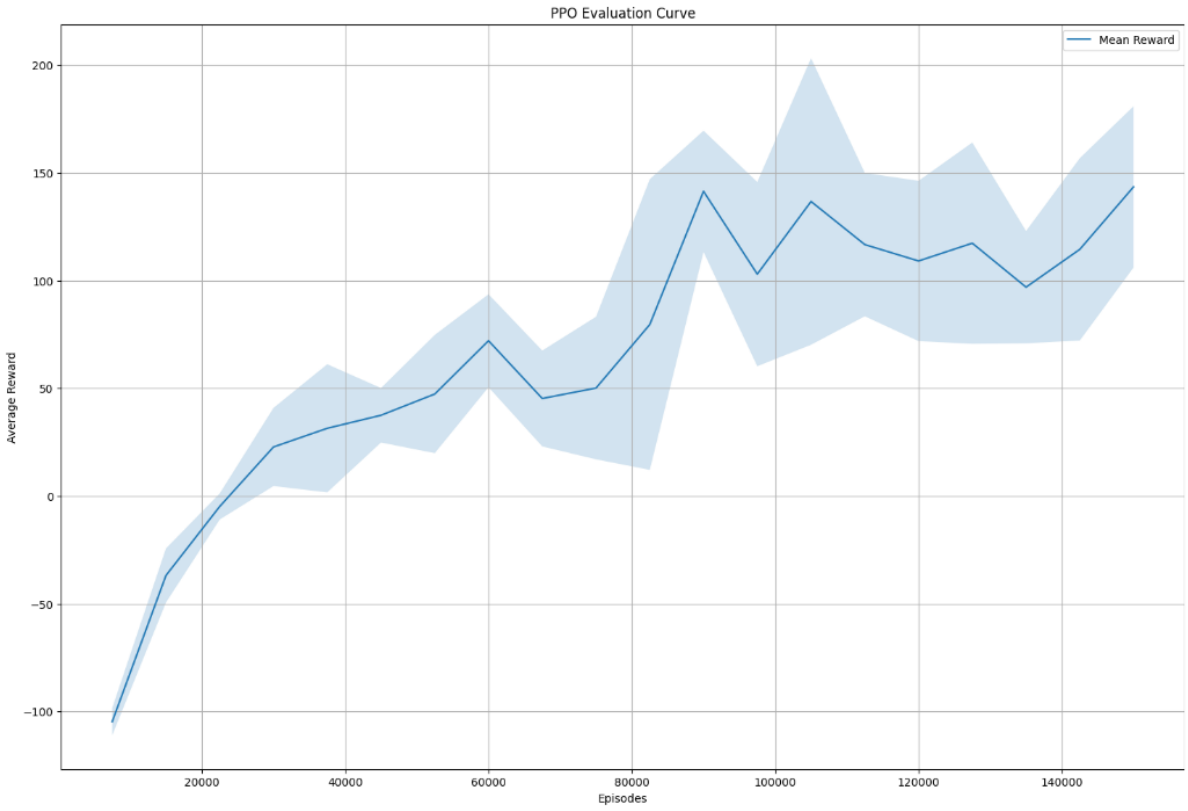
Experiments and Results

Lunar Lander:
We varied the following hyper-parameters:

T_horizon
K_epochs
batch_size

The average rewards was plotted

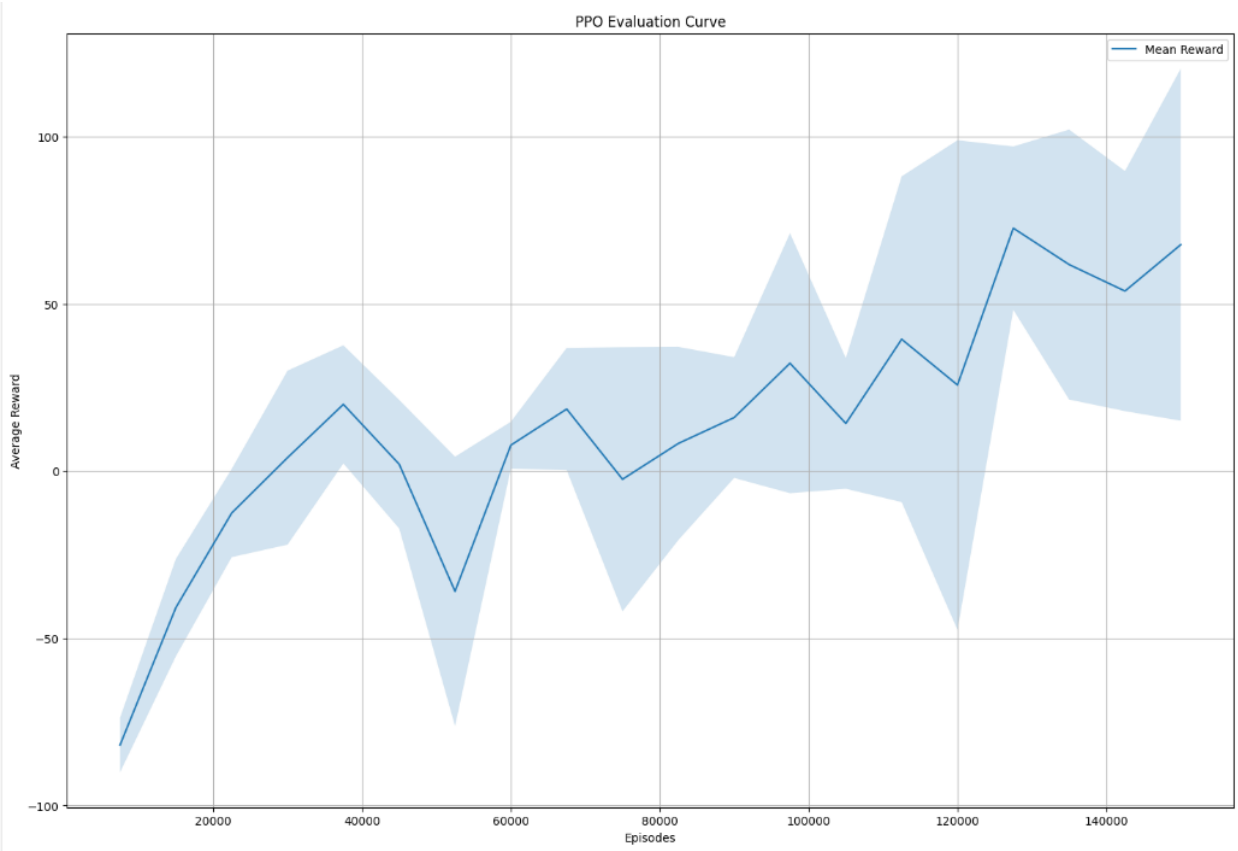
K_epochs = 8
T_horizon = 2048
batch_size = 32



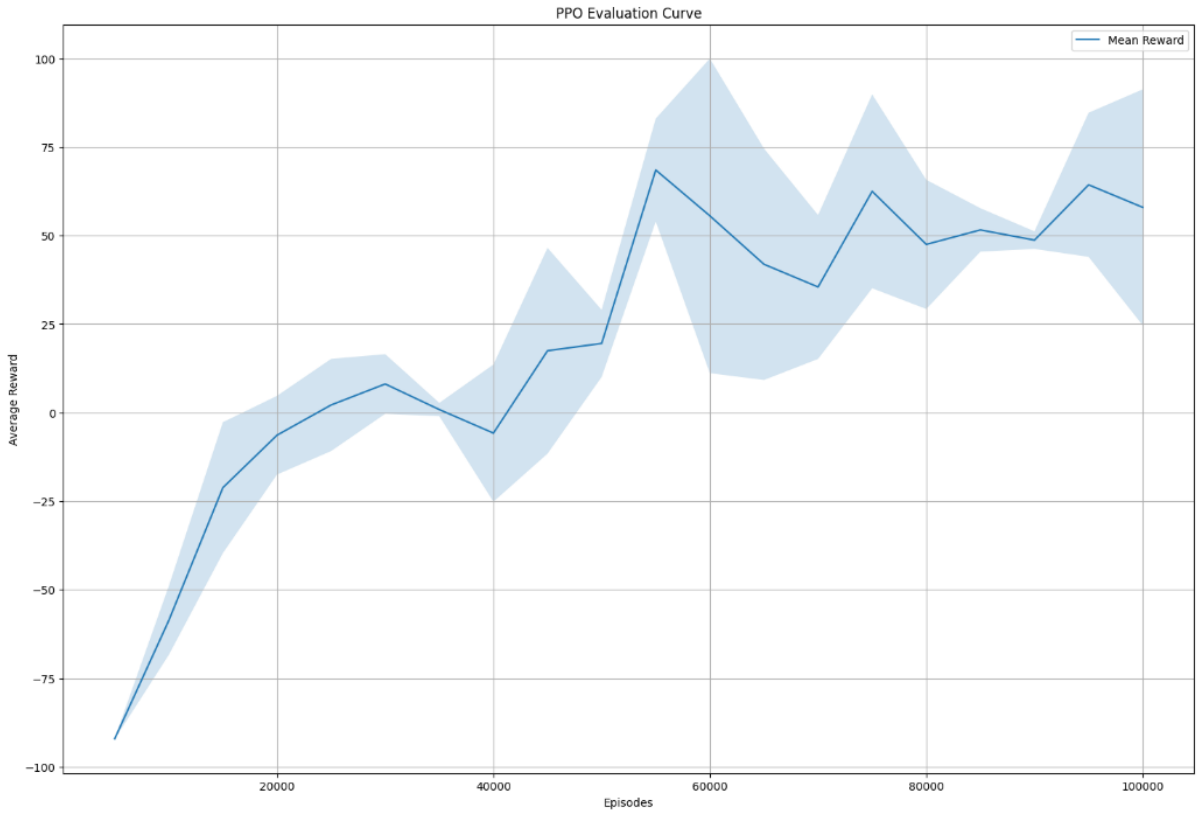
K_epochs = 24

T_horizon = 2048

batch_size = 32



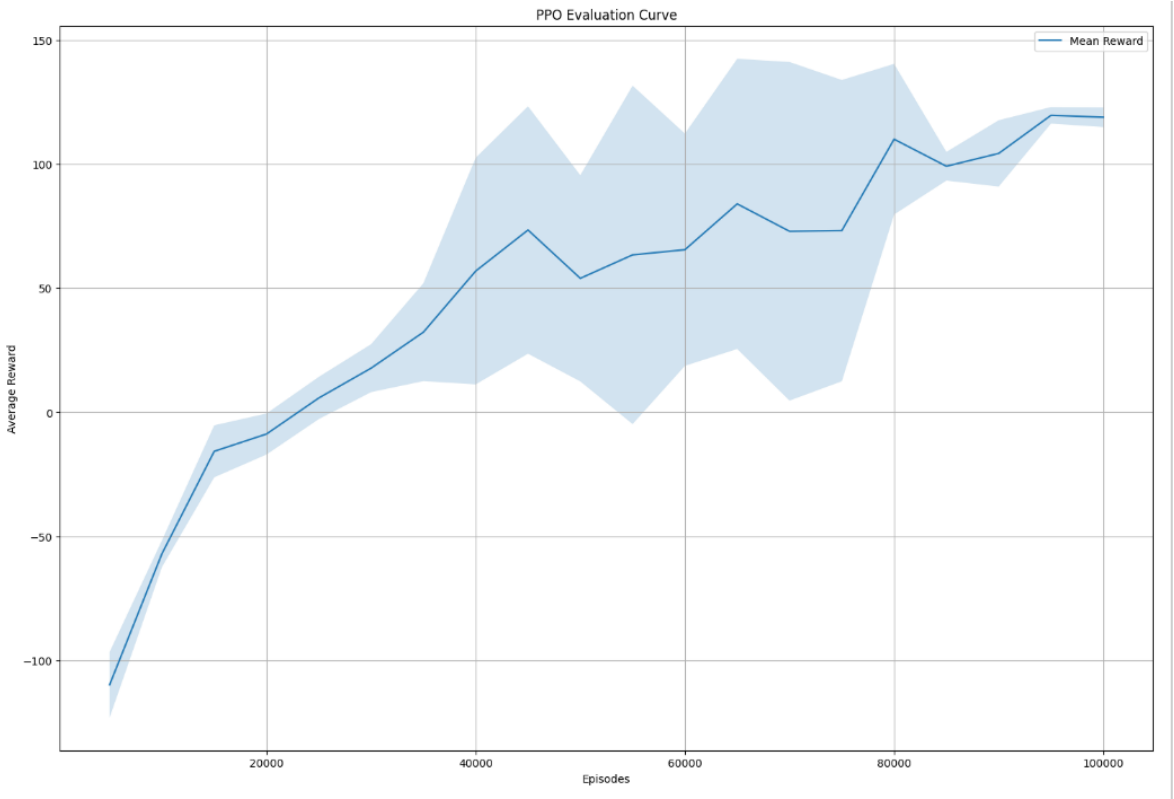
K_epochs = 40
T_horizon = 2048
batch_size = 64



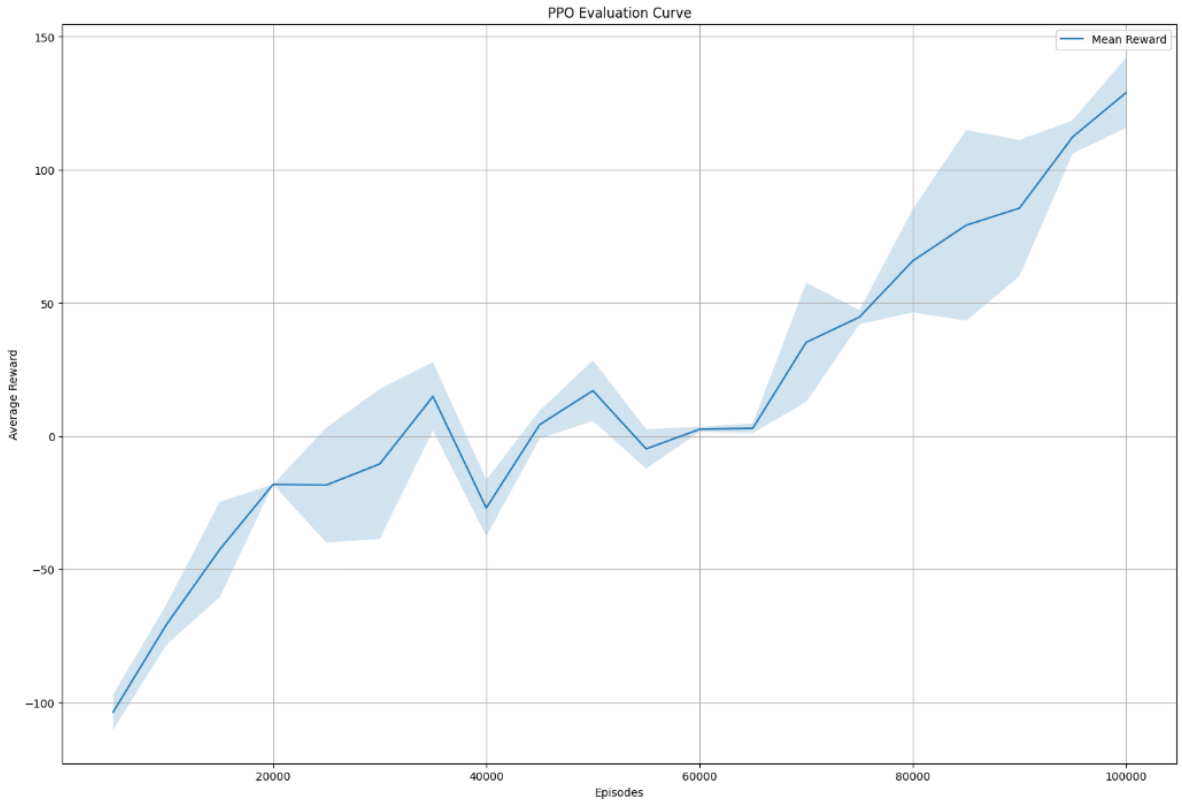
K_epochs = 32

T_horizon = 2048

batch_size = 64



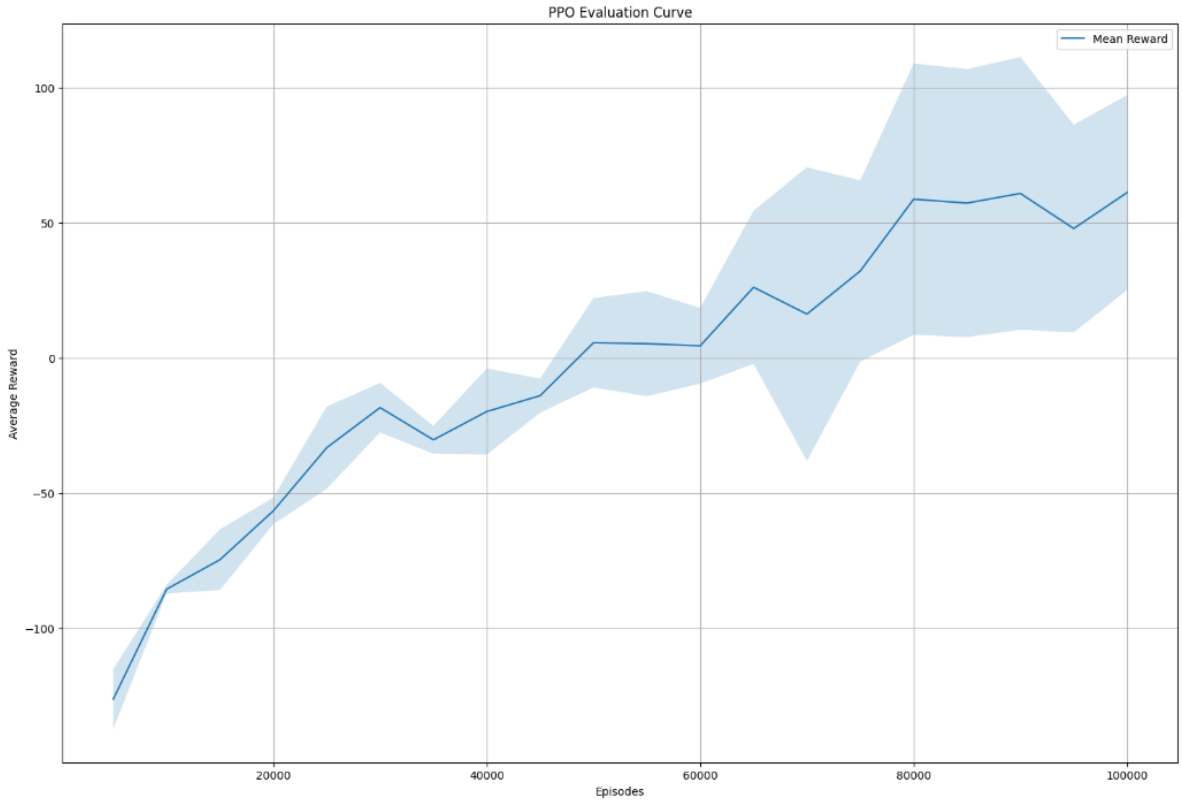
K_epochs = 32
T_horizon = 2048
batch_size = 32



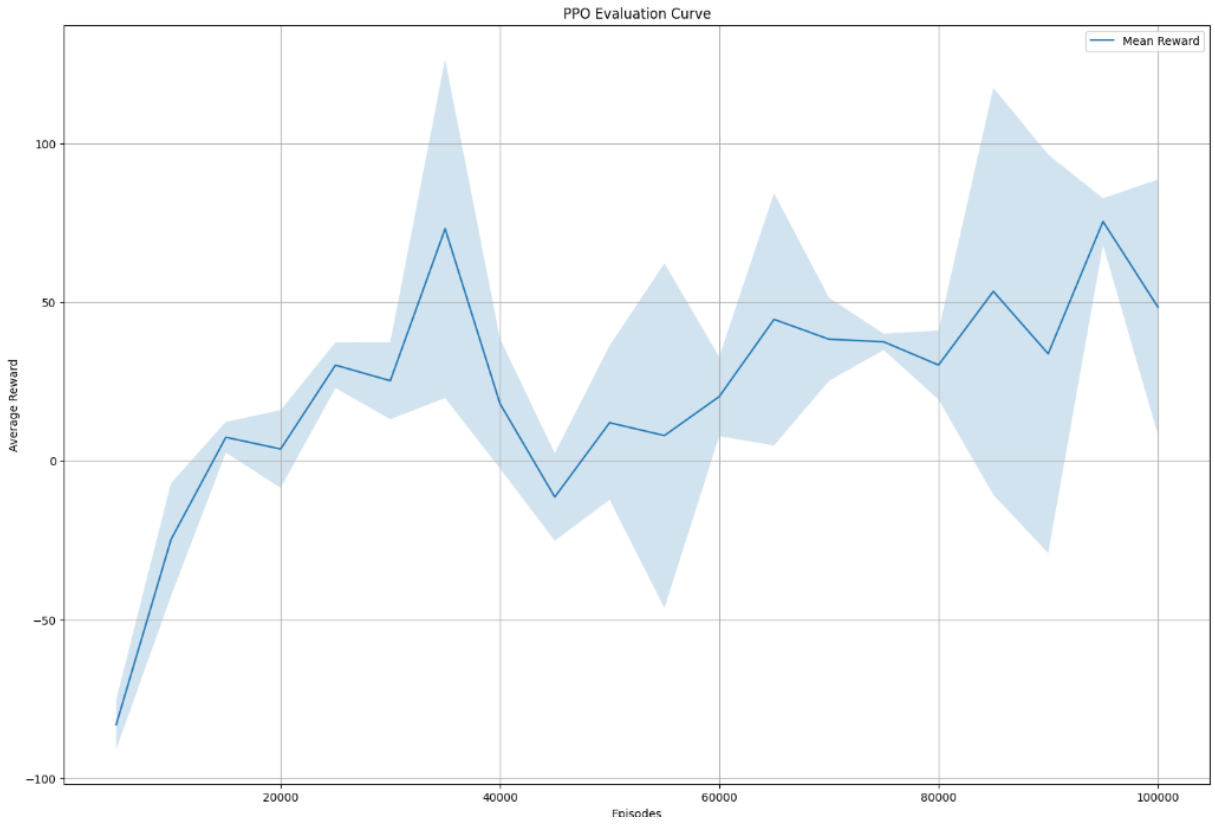
K_epochs = 32

T_horizon = 3072

batch_size = 32



K_epochs = 20
T_horizon = 1024
batch_size = 32



Out of all the experiments done with the three hyper-parameters, from the graphs The T_horizon=2048 and batch_size=32 worked best. The most confusing part of these experiments is that having epochs=8 produced good results reaching around 150, epochs=32 produced the best results, with the average rewards crossing 150. But, for epochs=24, which is in between the two, the rewards collected were just above 50 and this performed the worst. This confuses me, since there is no monotonicity in between the number of epochs.

When I tested with T_horizon and epochs constant and batch_size was changed, both gave almost similar average rewards, but the experiment with epochs=64 had higher variance in between. We could consider having higher variance in the middle; to be a good factor, this can be taken as a sign of higher exploration. But there was no proper conclusion made from these two results.

Now, when I vary the T_horizon, having 1024 or 3072, both performed inferior when compared to 2048. This might show that for Lunar Lander, 2048 is a good number for our agent to explore before making the PPO updates.

My assumption before these results was that, having smaller epochs and smaller T_horizon would work better than, since PPO is an on-policy algorithm and having higher T_horizon would collect a lot of data before actually making any update, which feels like an off-policy update. The smaller epoch size would ensure that we are not overfitting on the single batch of data, I am not sure if overfitting is a concept that applies to RL.
My thoughts on batch_size was it should be large enough to actually make sense of the data, if the batch size is very low then we would not have enough sample for the algorithm to make sense.

After my results, it seems to me that there is no direct relationship that could be underlined from my experiments but my idea was that all three are intricately convoluted, like rather than batch_size and T_horizon making sense separately, A value like T_horizon/batch_size would make some sense or something similar to it like, epochs*batch_size so that efficient use of our sample is done which could be tested when this is extended, but these are just thoughts.

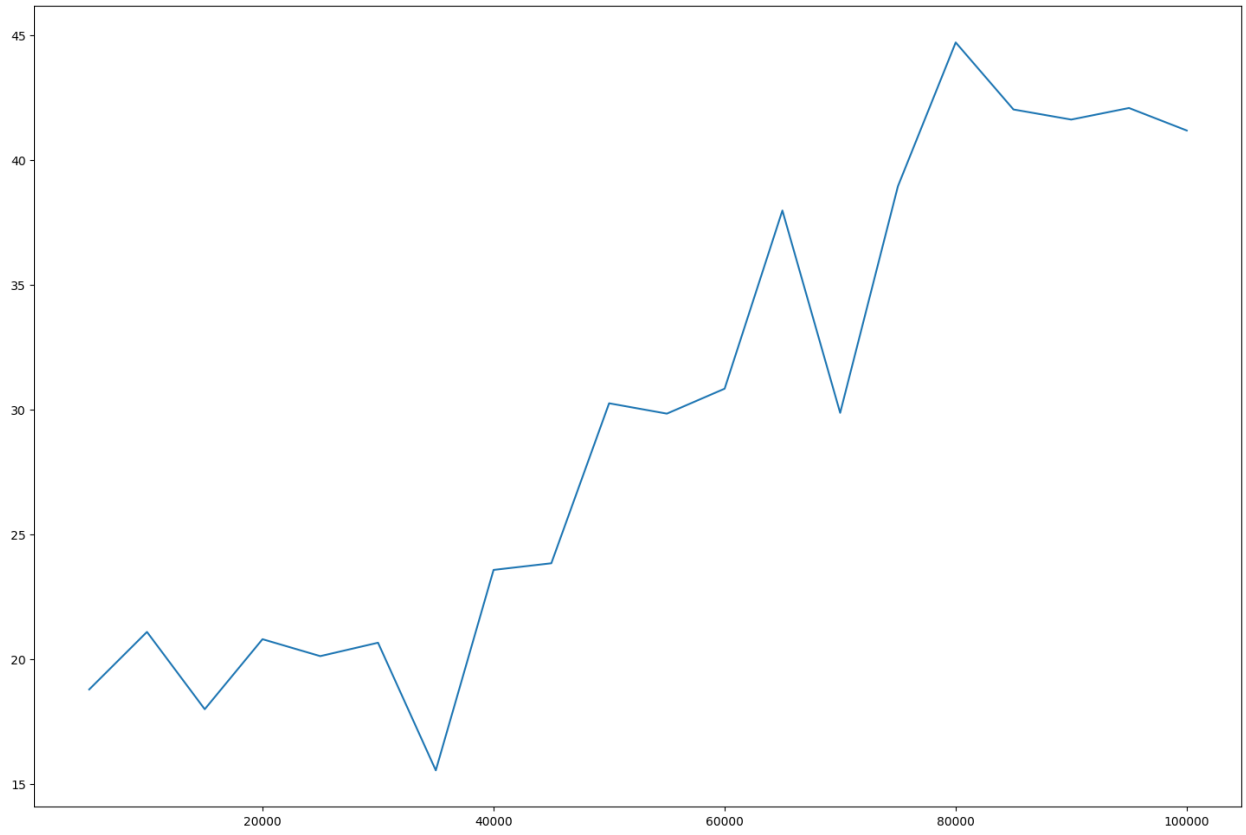
My goal was to breach the average rewards barrier of 200, since it was mentioned in the documentation that this is considered to be a success. My implementation's average rewards over multiple runs did not cross that however a single run touched 199 while during the next run with the same hyperparameters the average rewards was just above 50, meaning there still a lot of stochasticity left to uncover, to provide a generalized policy, since the initial conditions differ in each episode.

Swimmer:
The swimmer experiments were run on GPU in Google colab since it takes a lot of time and compute to finish a single test run.
Even in this experiment, we varied the same three hyper-parameters

K_epochs = 16

T_horizon = 3072

batch_size = 64



K_epochs = 20

T_horizon = 1024

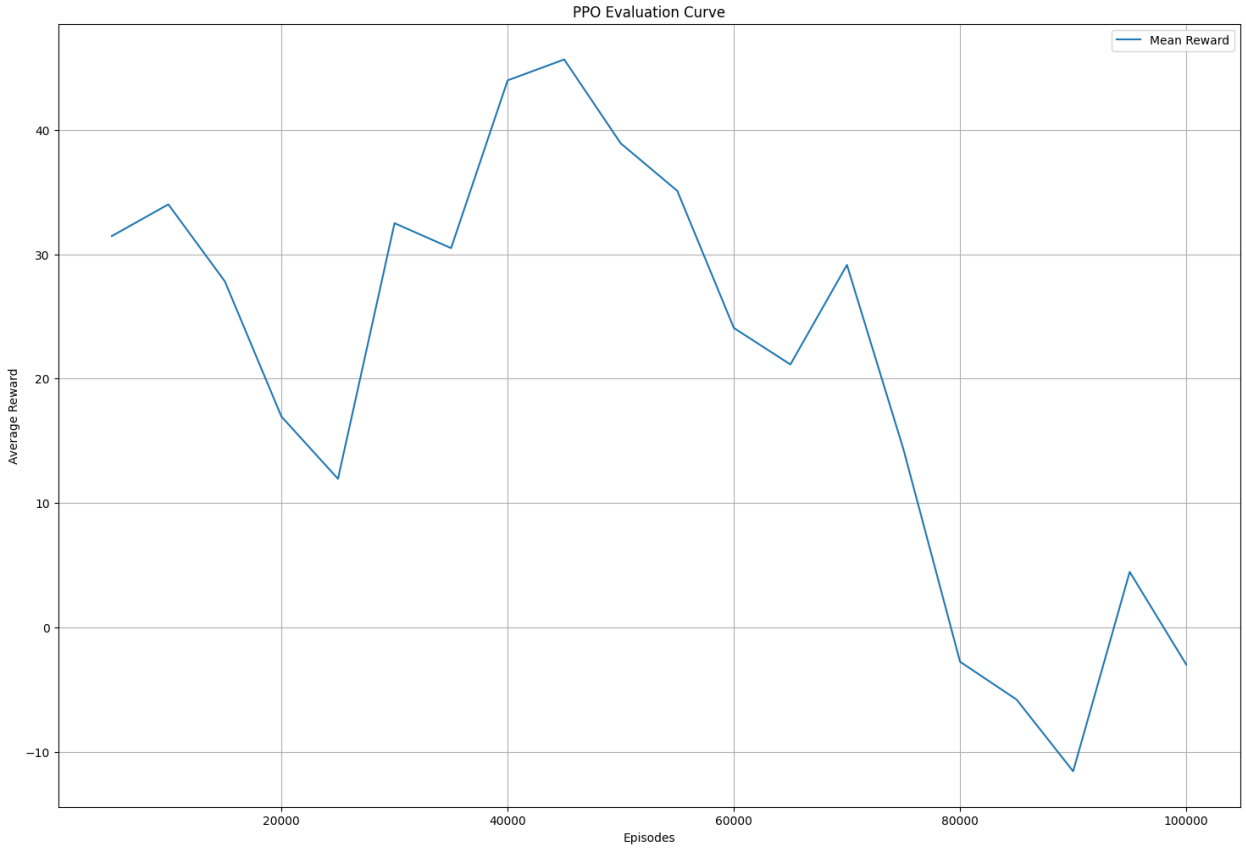
batch_size = 32



K_epochs = 30

T_horizon = 2048

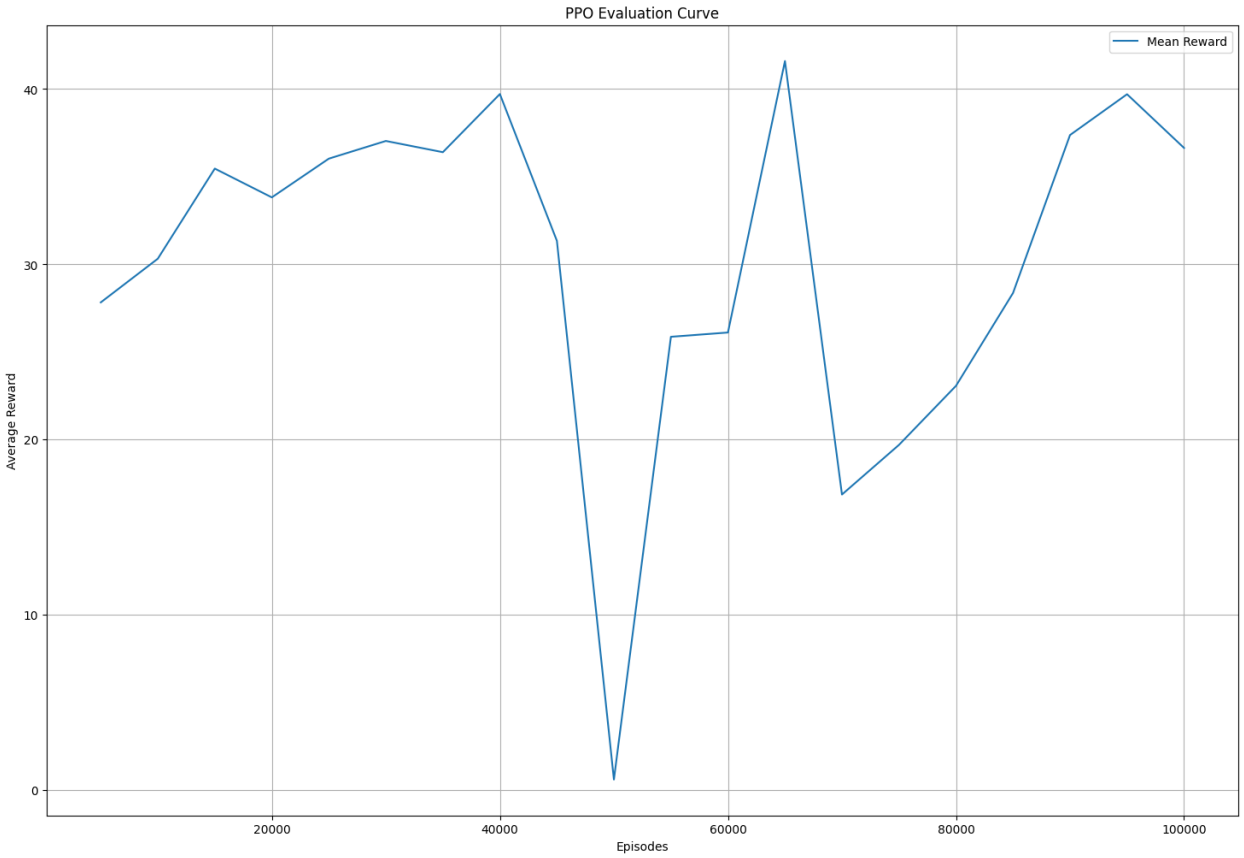
batch_size = 64



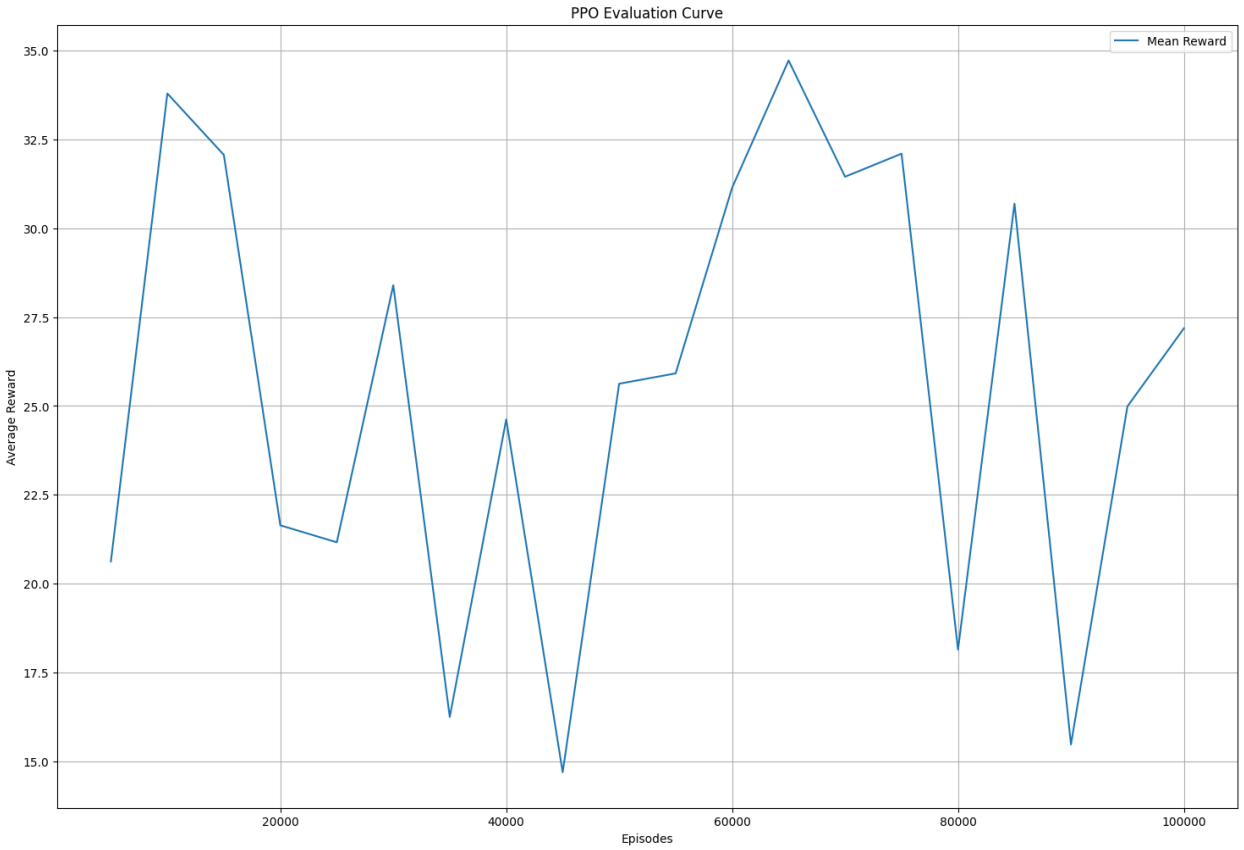
K_epochs = 16

T_horizon = 2048

batch_size = 32



K_epochs = 24
T_horizon = 2048
batch_size = 64



Having lower T_horizon performed poorly, with having a higher T_horizon, 3072 producing the best results. Among the 2048 tests with batch_size=64, having lower epochs worked better, but with little evidence to support that claim, since there is very high variance and the experiments were run only once due to the time issue. The attempts at running twice resulted in the runtime getting disconnected in between yielding no results. Instead of multiple experiments with 3072 as T_horizon, multiple attempts were done due the mention of 2048 in the 'original' for their testing. However, their test ran for 10 times more steps than the tests that ran here. So, it would have been better if I did not follow their hyper-parameters. In the 'original', PPO achieved over 100 as its average rewards, while my implementation barely crossed 40.

Difficulties:

The hyper-parameter tuning was difficult in this project. I could not make sense of the results to make a calculated guess as to which set would yield better results. The disconnected runtime in colab put me back since a lot of hours of runtime was wasted. The 'original' did not have a clear explanation of the effects the hyper-parameters would have or any thumb rule to help us choose. The paper by Costa Huang helped a lot in this front. The common problem faced by students in RL, the high computing requirements were present in my project too.

How would I tackle if I re-do it:

The followed the 'original' as a gold standard in implementing PPO and did not take note other resources present, this thwarted my progress because the small things I implemented from other approached really helped my algorithm work like the normalization of advantages, neural networks best practices like clipping the gradient.

I would try to implement a small helper function which would plot the graph after every run and save each image into my local. In this way, if I run my experiments 7 times and due to some external factors, the program stops while running the last iteration, I would still have my progress until then rather than not having anything.

I should have had a better way of conducting my experiments on hyper-parameters, I am now left with a graph for each test. Plotting everything in a single graph would have been more illustrative. This way, I had a tough time making sense of the results.

I would try and test the effect of having deeper neural nets and initializing the neural networks with proper weights like glorot initialization. This totally slipped my mind.

This project highlighted my inexperience in doing research or projects, I did not have a proper strategy in how I would come forward and deviated a lot from initial thoughts on how I would go on in this project, or how I would vary my hyper-parameters. I did not check multiple sources before starting and getting struck. If I re-do this project, I would want to have a proper idea/ strategy before starting my work.

Further Work:

I would like to explore more on my hunch/idea of the hyper-parameters batch_size, T_horizon and epochs being connected in a way that some operations would yield a constant and that constant matters in hyper-parameter tuning.

Following the best practices of neural nets, like proper initialization, dropout factor.

References:

Proximal Policy Optimization Algorithms by John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov OpenAI
https://costa.sh/blog-the-32-implementation-details-of-ppo.html
https://medium.com/@danushidk507/ppo-algorithm-3b33195de14a
https://spinningup.openai.com/en/latest/algorithms/ppo.html

Gen AI was used to verify and help in the syntax of pytorch and debugging some of the errors.